



Encontro 2

Histórico, Navegação e Desfazimento

0:00–0:20

Revisão Encontro 1

0:20–1:00

Histórico e inspeção

1:00–1:30

.gitignore na prática

1:30–2:40

Desfazimento

2:40–3:00

Quiz (10 perguntas)

```
git log --oneline --graph --all
```

Revisão rápida — o que vimos no Encontro 1

Git ≠ GitHub

Git é instalado na máquina. GitHub é a plataforma web.

Configuração inicial

user.name e user.email definem sua identidade em todos os commits.

```
$ git config --global user.name / user.email
```

Ciclo básico

Três estados: working directory → staging → repository.

```
$ git add . → git commit -m "msg"
```

Repositório remoto

Vinculamos o projeto local ao GitHub e fizemos o primeiro push.

```
$ git remote add origin <url> → git push
```



Lendo o histórico de commits — git log

```
$ git log
```

Log completo

Mostra autor, data, hash completo e mensagem. Útil para investigar detalhes de um commit específico.

```
$ git log --oneline
```

Log compacto

Hash abreviado + mensagem. Ideal para ter uma visão rápida do histórico. Use no dia a dia.

```
$ git log --oneline --graph
```

Log com grafo

Mostra a estrutura de branches visualmente com caracteres ASCII. Ótimo para entender ramificações.

```
$ git log --oneline --graph --all
```

Todas as branches

Inclui branches remotas e locais no grafo. Visão completa do repositório.

O que é um hash SHA-1?

a1b2c3d4e5f6789012345678901234567890abcd

hash SHA-1 completo — 40 caracteres hexadecimais únicos

É único

Dois commits diferentes nunca terão o mesmo hash. É a "impressão digital" do commit.

É gerado automaticamente

O Git calcula o hash com base no conteúdo dos arquivos, autor, data e mensagem.

Na prática, use só os 7 primeiros

O Git aceita o hash abreviado. Ex: a1b2c3d já identifica o commit com segurança.

Para que serve

Referenciar commits em comandos como git show, git revert, git reset e git checkout.



Inspecionando diferenças — git diff e git show

```
$ git diff
```

O que mudou (antes do add)

Compara o working directory com a staging area. Mostra linhas removidas em vermelho (–) e adicionadas em verde (+). Use antes de fazer git add.

```
$ git diff --staged
```

O que vai entrar no commit

Compara a staging area com o último commit. Útil para revisar exatamente o que será commitado antes de executar o git commit.

```
$ git show <hash>
```

Detalhes de um commit específico

Mostra o autor, data, mensagem e o diff completo de um commit. Substitua <hash> pelo hash do commit (os 7 primeiros caracteres bastam).

```
$ git show HEAD
```

Detalhes do último commit

HEAD aponta sempre para o commit mais recente da branch atual. Um atalho muito útil no dia a dia.



.gitignore — o que não deve ir para o repositório

O que NÃO commitar

node_modules/

dependências (pesadas, reinstalar é fácil)

.env

senhas, tokens, chaves de API

.DS_Store

arquivos do sistema macOS

***.log**

arquivos de log

dist/ / build/

arquivos compilados/gerados

.vscode/

configs pessoais do editor

Como usar

```
# arquivo .gitignore
node_modules/
.env
*.log
dist/
.DS_Store
```

- 1 Criar o arquivo .gitignore na raiz do projeto
- 2 Adicionar um padrão por linha
- 3 Commitar o .gitignore junto com o projeto

gitignore.io — gerador automático por linguagem

E se eu já comitei algo que não deveria?

Situação comum: commitou o node_modules/ ou o .env antes de criar o .gitignore

Remover do rastreamento (mas manter o arquivo localmente)

1

```
$ git rm --cached nome-do-arquivo  
git rm -r --cached nome-da-pasta/
```

O arquivo sai do Git mas continua no seu computador. O -r é necessário para pastas.

Adicionar ao .gitignore

2

```
$ # dentro do arquivo .gitignore  
nome-do-arquivo  
nome-da-pasta/
```

Para garantir que não seja adicionado novamente por engano.

Comitar a remoção

3

```
$ git add .gitignore  
git commit -m "Remove arquivo sensível do histórico"
```

Registra a mudança. Atenção: se o arquivo continha senhas e foi publicado no GitHub, troque as credenciais imediatamente.



Desfazimento — quando usar cada abordagem

```
git revert <hash>
```

SEGURO

Desfazer um commit que já foi para o GitHub

Cria um novo commit que desfaz as mudanças. Histórico é preservado.

Ideal em equipe: não reescreve histórico compartilhado. Transparente — todos veem o que foi desfeito.

```
git reset --soft HEAD~1
```

LOCAL

Desfazer o último commit mas manter os arquivos preparados

Remove o commit, mas mantém as mudanças na staging area. Você pode reescrever a mensagem.

Use para corrigir a mensagem de commit ou juntar mudanças antes de commitar novamente.

```
git reset --hard HEAD~1
```

PERIGOSO

Apagar completamente o último commit e todas as mudanças

Remove o commit e descarta todas as alterações. Não tem desfazer.

Use apenas localmente, antes de fazer push. Dados perdidos não voltam.

```
git restore <arquivo>
```

LOCAL

Descartar mudanças em um arquivo ainda não commitado

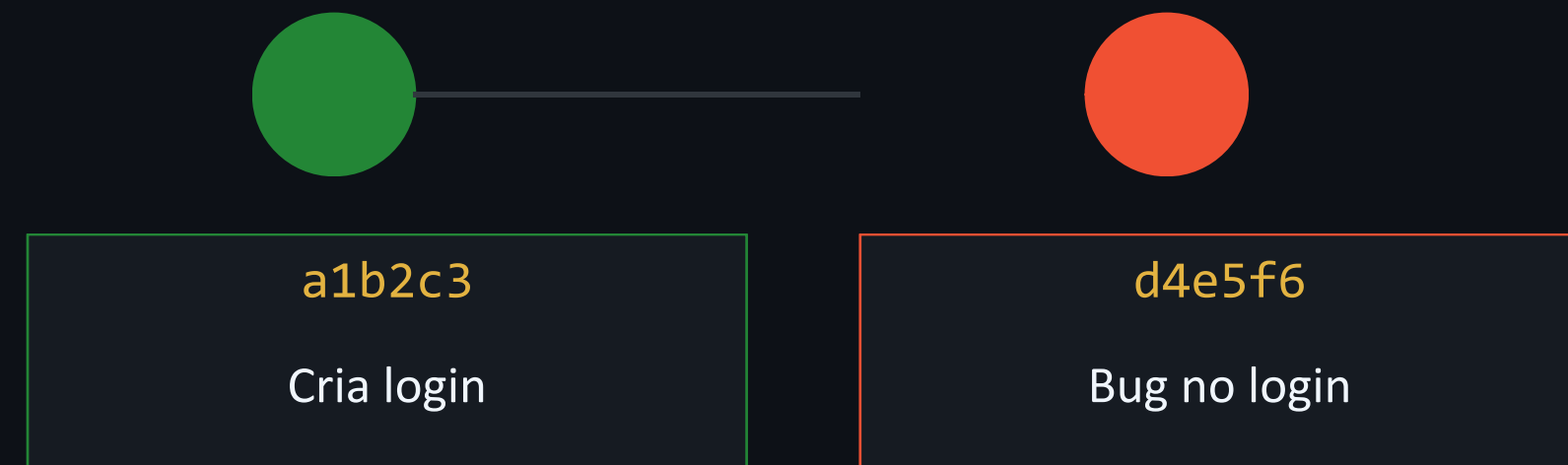
Volta o arquivo ao estado do último commit. Mudanças não preparadas são perdidas.

Antigo: git checkout -- arquivo. O git restore é a forma moderna e mais clara.

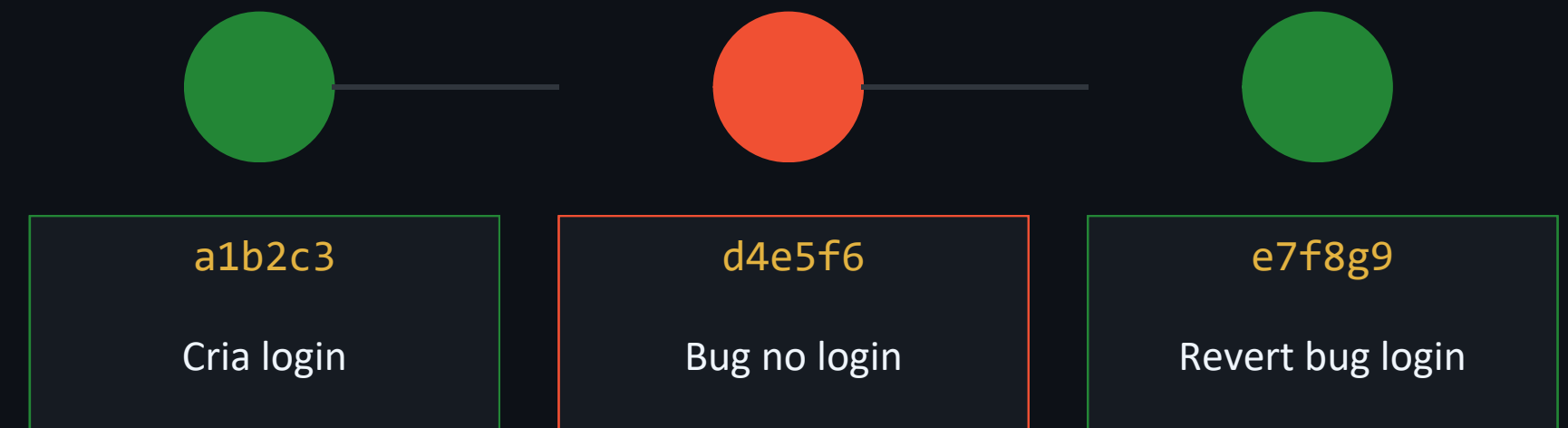
git revert — desfazendo com segurança

Preferido quando o commit já foi para o GitHub (repositório compartilhado)

Antes do revert



Depois do revert



Passo a passo

1

```
$ git log --oneline
```

Identificar o hash do commit que quer desfazer

2

```
$ git revert d4e5f6
```

Git abre o editor para a mensagem do revert commit

3

```
$ git log --oneline
```

Verificar que um novo commit de revert foi criado

4

```
$ git push
```

Enviar o revert para o GitHub (seguro!)



git reset — use com cuidado

Reescreve o histórico — use apenas localmente, antes de fazer push

--soft

```
$ git reset --soft HEAD~1
```

Desfaz o commit, mantém arquivos na staging

É como se o commit nunca tivesse existido, mas todas as mudanças ainda estão preparadas para um novo commit. Use para reescrever a mensagem ou ajustar o conteúdo antes de commitar novamente.

Commit removido · Staging preservada · Arquivos ok

--mixed

```
$ git reset HEAD~1
```

Desfaz o commit e a staging (padrão)

As mudanças voltam para o working directory (não commitadas, não preparadas). Você decide o que vai add e commitar novamente.

Commit removido · Staging limpa · Arquivos ok

--hard

```
$ git reset --hard HEAD~1
```

⚠ Apaga tudo — commit e alterações

Remove o commit e descarta todas as mudanças nos arquivos. NÃO há desfazer. Dados perdidos não voltam. Use apenas quando tem certeza absoluta.

Commit removido · Staging limpa · Arquivos apagados

git restore — descartar alterações não commitadas

Use quando você editou um arquivo mas quer voltar ao estado do último commit

```
$ git restore arquivo.txt
```

Descartar mudanças em um arquivo

Volta o arquivo ao estado do último commit. Mudanças não preparadas são perdidas. Cuidado: não tem desfazer.

```
$ git restore .
```

Descartar todas as mudanças locais

Reverte todos os arquivos modificados. Use quando quer limpar tudo e começar do estado do último commit.

```
$ git restore --staged arquivo
```

Tirar da staging (manter edição)

Remove o arquivo da staging area mas mantém as mudanças no working directory. Equivale ao antigo git reset HEAD arquivo.

Exercício — Viagem no tempo

Prática guiada — todos fazem ao mesmo tempo

1

Criar 4 commits no projeto

Adicione um arquivo diferente em cada commit com uma mensagem descritiva.

```
$ echo "v1" > versao.txt && git add . && git commit -m "Versão 1"
```

2

Ver o histórico

Confirmar que os 4 commits aparecem.

```
$ git log --oneline
```

3

Estragar o projeto de propósito

Delete um arquivo importante e faça um commit com a 'bagunça'.

```
$ rm versao.txt && git add . && git commit -m "Ops, deletei"
```

4

Desfazer com revert

Reverter o commit ruim com segurança, mantendo o histórico.

```
$ git revert HEAD
```

5

Confirmar que voltou ao normal

Verificar que o arquivo voltou e o histórico está completo.

```
$ git log --oneline && ls
```

Resumo dos comandos — Encontro 2

Histórico e inspeção

```
git log
```

histórico completo

```
git log --oneline
```

histórico compacto

```
git log --oneline --graph --all
```

com grafo de branches

```
git diff
```

mudanças antes do add

```
git diff --staged
```

mudanças após o add

```
git show <hash>
```

detalhes de um commit

.gitignore

```
.gitignore
```

arquivo na raiz do projeto

```
git rm --cached <arq>
```

para de rastrear o arquivo

```
git rm -r --cached <pasta>
```

para de rastrear a pasta

Desfazimento

```
git revert <hash>
```

seguro — cria novo commit

```
git reset --soft HEAD~1
```

desfaz commit, mantém staging

```
git reset HEAD~1
```

desfaz commit e staging

```
git reset --hard HEAD~1
```

⚠ apaga tudo, sem volta

```
git restore <arquivo>
```

descarta mudanças locais

```
git restore --staged <arq>
```

tira da staging

```
git commit -m "Encontro 2 concluído!"
```


ENCONTRO 02

FINALIZADO!



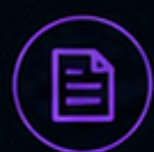
O QUE APRENDEMOS HOJE:



Histórico de commits com `git log`



Inspeção de alterações com `git diff` e `git show`



Uso do `.gitignore` na prática



Desfazimento com `git revert`, `git reset` e `git restore`



PRÓXIMO ENCONTRO:

**BRANCHES, MERGE
E REPOSITÓRIOS REMOTOS**



Muito bom trabalho!

Até o próximo encontro!